

Software Requirements Specification

for

fileManager

Prepared by Team12

Koo Bonjoon (20240032)

Lee Hohyung (20240596)

Lee Junseo (20240857)

Table of Contents

- 1 Introduction
 - 1.1 Purpose
 - 1.2 Document Conventions
 - 1.3 Intended Audience and Reading Suggestions
 - 1.4 Product Scope
 - 1.5 References
- 2 Overall Description
 - 2.1 Product Perspective
 - 2.2 Product Functions
 - 2.3 Operating Environment
 - 2.4 Design and Implementation Constraints
- 3 External Interface Requirements
 - 3.1 User Interfaces
 - 3.2 Hardware Interfaces
 - 3.3 Software Interfaces
- 4 System Features
 - 4.1 Search
 - 4.2 Categorization
 - 4.3 Personal Setting
- 5 Non-functional Requirements
 - 5.1 Performance Requirements
 - 5.2 Reliability
 - 5.3 Security
 - 5.4 User convenience
- 6 Appendix
 - 6.1 Flow Charts

1 Introduction

1.1 Purpose

The purpose of this paper is to define the functional and non-functional requirements of "fileManager", a file search and classification system that combines high-speed keyword search (Quick Search) and natural language-based semantic search (Smart Search) in a Windows environment.

1.2 Document Conventions

Term	Description
Smart Search	Search method that identifies context and meaning and protrudes results
Quick Search	Search methods supported by information such as the name of a pre-indexed file
Categorization	Software internal classification based on metadata and content of files
USN Journal	Journal that records file changes history for an NTFS volume
Overlay UI	Search window interface that appears above the existing task screen when entering specific hotkey

1.3 Intended Audience and Reading Suggestions

This document is intended for the team that chooses this project for their development project including developers, UI/UX designers and testers. There are 6 chapters in this document: 1. Introduction, 2. Overall Description, 3. External Interface Requirements, 4. System features, 5. Non-functional Requirements and 6. Appendix. All readers are encouraged to read chapters 1,2(Introduction, Overall Description) to understand the overall structure of this project and navigate the sections based on their specific roles.

Developers: It suggested reading all chapters, with extra attention on Chapter4 to understand requirements for each feature as well as Chapter5.

UI/UX Designers: User Interface in Chapter3 and stimulus/response sequences in Chapter4 Would be key sections.

Testers: Utilize functional requirements for each feature in Chapter 4 to build test cases.

1.4 Product Scope

The software aims to gain faster access to the files it wants when working on a computer. Therefore, it must provide the following features:

- 1.4.1 **Quick search:** Provides instant search results based on file name, like "Everything".
- 1.4.2 **Smart search:** Find the file by analyzing complex intentions such as "a software engineering paper downloaded a week ago".
- 1.4.3 **Accessibility:** Supports instant UI calls via hotkeys, such as Spotlight on macOS
- 1.4.4 **Personal setting:** The user must be able to specify several factors directly, including the installation path of the app, the storage location of the DB, the extension of the file to be targeted by smart search, and the path of the folder.

1.5 References

1.5.1 file search program: Everything

1.5.1.1 Everything is a program that quickly searches for files on a PC. By utilizing the file system index, it provides faster search capabilities than the default Windows File Explorer. Our Quick Search feature refers to this.

1.5.1.2 <https://www.voidtools.com/ko-kr/support/everything/>

1.5.2 file organization and automation tool: Tidrop

1.5.2.1 <https://apps.microsoft.com/detail/9pmhcnj9tx4f?hl=ko-KR&gl=KR>

2 Overall Description

2.1 Product Perspective

fileManager is a desktop file search and management software designed for the Windows environment. Rather than replacing the existing Windows File Explorer, this system is designed as a complementary tool to assist with file searching and classification.

The main functions consist of name-based **Quick Search**, natural language-based **Smart Search**, and automatic file **Categorization**. The system is composed of a backend engine that performs file indexing and categorization, and a frontend UI that provides search and configuration features.

The backend engine interfaces with the Windows file system to detect file creation and modification events, analyzing file metadata and content to store them in a local database. The frontend UI includes an **Overlay UI**—which can be invoked instantly via global hotkeys—and a main application mode.

Additionally, the system automatically analyzes files within monitored directories to generate categorization information such as subjects, projects, and document types, which is then used to improve the accuracy of Smart Search. Users can manually input additional keywords for specific files, and the system reflects these to update the categorization information. As a core principle, all functions are performed within the local environment rather than on an external server.

2.2 Product Functions

2.2.1 Search

Users can call up the overlay UI through hotkey, and the search must proceed with the bottom two items.

2.2.1.1 Quick search

Through pre-indexed data, a search is conducted for the name of the file, the last modification date, or the custom tag to show the results.

2.2.1.2 Smart search

The software identifies some of the contents of the file in advance, vectorizes it, and stores it in the DB. Based on the stored vectors, the syntax entered by the user is analyzed to show highly relevant files as search results.

2.2.2 Categorization

The file categorization system automatically analyzes files in folders that you specify and categorize them through specific criteria. This does not change the path of an actual file, but rather helps the software remember it and then helps with the search function.

2.2.3 personal setting

Upon installation, users can specify the application's installation path, the DB storage location, and the target folders and file extensions for Smart Search. These settings (DB location, target folders, and extensions) must be modifiable later. Furthermore, users must be able to customize the UI appearance as needed.

2.3 Operating Environment

2.3.1 **OS:** Windows 10/11

2.3.2 **Minimum specification:** over 100MB of free disk space for index storage and over 8GB of RAM for natural language processing.

2.4 Design and Implementation Constraints

2.4.1 **Variable Naming:** All code must use CamelCase, and variable names must clearly convey their meaning.

2.4.2 **Privacy:** To ensure security, all user file lists and search queries must be processed locally and never transmitted to an external server.

3 External Interface Requirements

3.1 User Interface

3.1.1 Setup

The Setup process guides the user through the installation of the application. It allows users to specify the following configurations:

- **App Installation Path:** The directory where the executable files will be stored.
- **Database Path:** The location where the index and vector databases will be saved.
- **Smart Search Scope:** Selection of specific file extensions and target directories for semantic analysis.
- **Categorization Targets:** Identification of directories where automatic file classification will be applied.

3.1.2 Overlay UI

The Overlay UI is a specialized search interface that can be invoked via global hotkeys (e.g., Alt + Space).

- **Search Input:** Upon invocation, a simple input field appears in the center of the screen.
- **Dual Result Display:** Search results are displayed in a split-view format. **Quick Search** results appear on the left, while **Smart Search** results appear on the right.
- **Dynamic Layout:** While Smart Search is processing, the Quick Search area expands to fill more space. If no Quick Search results are found, the Smart Search area takes priority in the layout
- **Dismissal:** Users can close the Overlay UI by pressing the Esc key or clicking outside the interface boundaries

3.1.3 Setting

This page allows users to review application information and modify configurations. It consists of five primary tabs:

- **General:** Manages basic app information, including the installation path, DB path modification, and global hotkey settings.
- **Search Scope:** Manages the file extensions and folders targeted for Smart Search

- **Indexing Status:** Displays the current progress and status of system indexing and vectorization.
- **Theme & Appearance:** Allows users to customize the visual theme (e.g., Light/Dark mode).
- **App Information:** Provides version details and developer information.

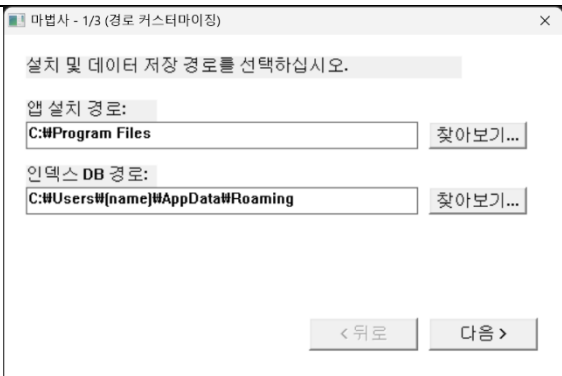
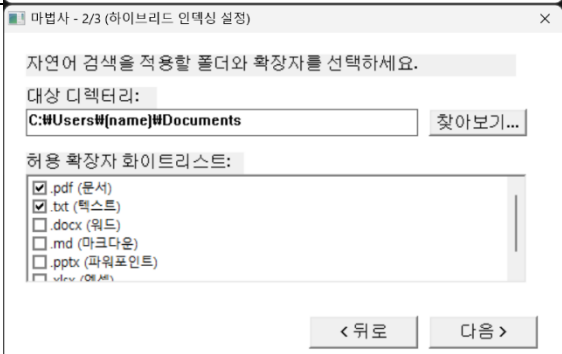
3.1.4 Custom Keywords

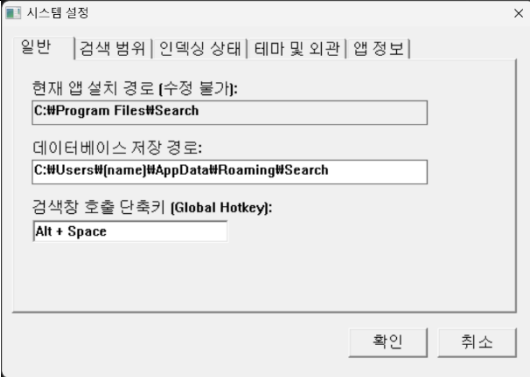
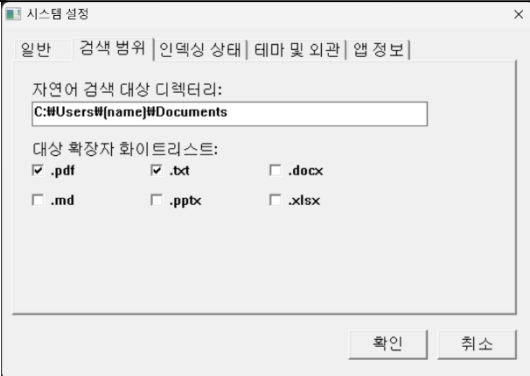
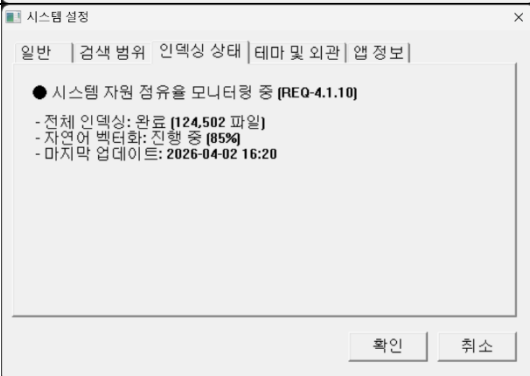
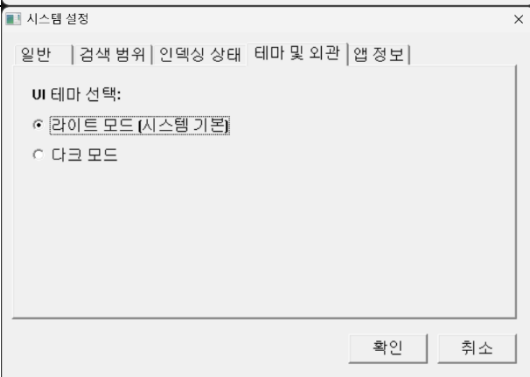
This interface allows users to manually assign keywords to specific files, supplementing the automatic results from the **Categorization** engine.

- Users can select a file and enter custom keywords, which are then saved to improve metadata and search accuracy.

3.1.5 References

This is a reference for UI development. It doesn't have to be in the same form as this material.

Setup1			
Setup2			

Setting2			
Setting3			
Setting4			
Setting5			
Setting6			

Setting7	시스템 알림  설정이 저장되었습니다. 변경된 범위에 대해 벡터화를 즉시 재수행합니다. (REQ-4.1.11) 확인
KeyWord1	Add Keywords Selected File Browse... File Name Annual_Report_2026.pdf File Path /Documents/Reports/Finance/ File Type PDF Document Last Modified April 3, 2026 2:34 PM Existing Keywords project × documentation × important × Add New Keyword Enter a keyword Add Added keywords are used to improve file classification metadata and natural language search accuracy. Cancel Save

3.2 Hardware Interface

- 3.2.1 Storage Device: The system supports interfaces to access the USN Journal on HDDs and SSDs formatted with the NTFS file system.
- 3.2.2 Processor (CPU/GPU): The system efficiently allocates resources for vector operations and embedding model execution during natural language query processing.
- 3.2.3 Memory: The system manages RAM allocation to handle large-scale indexing data, potentially utilizing in-memory database structures.

3.3 Software Interface

- 3.3.1 Windows API: Calls system APIs to detect real-time changes in the file system.
- 3.3.2 Local NLP Engine: Establishes data exchange interfaces with local AI models (e.g., ONNX Runtime, Llama.cpp) to interpret natural language queries.
- 3.3.3 Database Interface: Communicates with local databases (such as SQLite or NoSQL formats) to store and retrieve indexed data.

4 System Features

4.1 Search

4.1.1 Description

File Search is a core feature that enables users to instantly find the files they need via an Overlay UI. It provides a hybrid experience, combining **Quick Search** which utilizes a local database index with **Smart Search**, which understands context through text vectorization.

When a user searches for a file, the system executes both methods simultaneously. The results screen is divided into two sections: Quick Search on the left and Smart Search on the right.

Initially, the system presents a list of results via Quick Search for immediate access. Once Smart Search is complete, the Quick Search view adjusts its width to reveal the Smart Search results alongside it.

Quick Search results display the filename, path, and the last modified date. Smart Search results display only the filename.

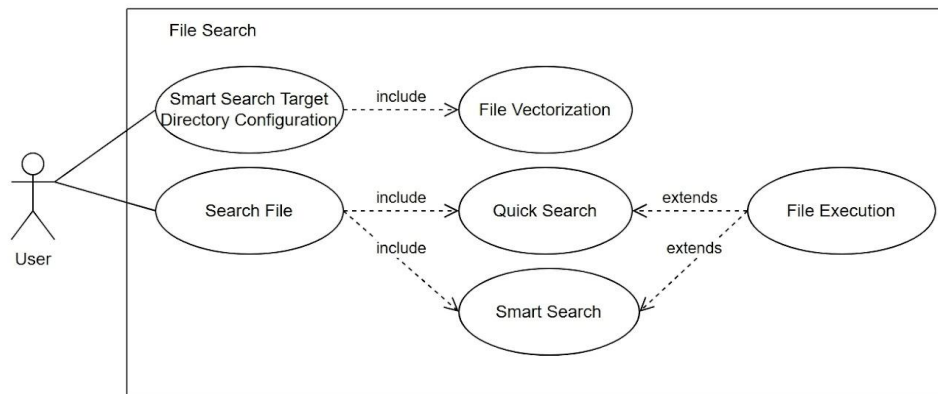
4.1.2 Stimulus/Response sequences

4.1.2.1 scenario 1: Invoking Overlay UI and search file

- A. The user presses a global hotkey (e.g., Alt + Space).
- B. The system activates the Overlay UI in the center of the screen.
- C. The user types a search query using the keyboard.
- D. The system queries the local index (DB) and outputs a list of files containing the search term in their filenames within 0.1 seconds.
- E. The system parses the search term and extracts the time condition (e.g., "yesterday"), file types (e.g., PDF for "paper"), and core semantics.
- F. Based on the extracted conditions, the system performs a vector similarity search and filtering, prioritizing the most relevant results, and outputs them in sequence below the already displayed file list.
- G. The user clicks on a file from the search results.
- H. The system executes the clicked file.

4.1.3 Functional requirements

- 4.1.3.1 **[Interface Invocation]**: The system shall run in the background and expose the search interface on the top-most layer upon the user's input of a designated global shortcut.
- 4.1.3.2 **[High-Speed Index Query]**: Upon keyword input, the system shall query the pre-built local database index and return matching results.
- 4.1.3.3 **[Time Condition Parsing]**: The system shall extract time-related expressions (e.g., "yesterday", "last week") from the input natural language string and convert them into the file's creation/modification date filter (last_modified) values.
- 4.1.3.4 **[File Type Parsing]**: The system shall extract words designating file types (e.g., "paper/thesis", "photo") from the input natural language string and convert them into an array of related extensions (e.g., [.pdf], [.jpg, .png]).
- 4.1.3.5 **[Semantic-Based Search]**: If simple keyword matching using file name, time condition and file type yields insufficient results, the system shall calculate the similarity between the embedding vector of the file content and the search term vector to return semantically related files.
- 4.1.3.6 **[Exception Handling - Empty Results]**: If no files match the search conditions, the system shall output an information message stating "No results found" instead of displaying a blank screen.
- 4.1.3.7 **[Hybrid indexing]**: The system shall perform keyword-based USN Journal indexing for the entire file system, but vector data for Smart Search shall only be generated within the scope specified by the user.
- 4.1.3.8 **[Dual Search Result Output]**: The system shall split the UI left and right to independently output Quick Search results on the left and Smart Search results on the right.



4.2 Categorization

4.2.1 Description

The file categorization system automatically analyzes newly stored files and internally classifies them into categories such as subject, project, and file type. The system detects newly created or modified files in user-specified directories in real time. It then analyzes metadata such as file name, extension, creation date, modification date, and storage path. For document files whose text can be extracted, the system additionally analyzes the title or body content to better understand the category of the file.

This feature is especially useful for users who repeatedly download large numbers of lecture materials, assignment files, papers, project documents, and image resources. For example, when a user downloads a PDF related to a machine learning course, the system can internally categorize the file as course-related material based on the file name and content analysis. Project documents, presentation files, and image files can also be categorized in a similar manner.

In addition, the system does not rely solely on automatic analysis results. It also allows users to manually add custom keywords to specific files. User-added keywords are stored together with the file's categorization information and are later used to refine categorization

results and improve Smart Search accuracy. This allows the system to reflect user intent or personal context that may not be fully captured through automatic categorization alone.

The categorization information is stored internally and is used later to improve the accuracy of Smart Search. Therefore, instead of directly viewing categorization results, users can more easily locate categorized files through Smart Search queries such as "the operating systems assignment I downloaded yesterday" or "the machine learning paper I read last week."

This feature has a high priority because it is one of the core functions of the service. If it does not operate properly, the accuracy and efficiency of Smart Search may decrease significantly, reducing the core value of the system.

4.2.2 Stimulus/Response sequences

4.2.2.1 scenario 1: Automatic categorization of a new file

- A. The system detects a file creation or modification event.
- B. The system collects metadata such as file name, path, extension, creation date, and modification date.
- C. If the file is in a supported document format, the system analyzes the title or content of the file.
- D. Based on the analysis result, the system assigns categorization information to the file and stores it in the database.

4.2.2.2 scenario 2: Modification of an existing file

- A. The system detects that an existing file has been modified.
- B. The system compares the existing categorization information with the latest metadata and file content, and updates the categorization information in the database if necessary.

4.2.2.3 scenario3: Deletion of a file

- A. The system detects that an existing file has been deleted or moved out of the watched directory.
- B. The system deletes the corresponding categorization information

from the database.

4.2.2.4 scenario4: User adds custom keywords

- A. The user adds one or more keywords to a specific file.
- B. The system stores the user-input keywords together with the existing categorization information of the file.
- C. The system updates the file's internal categorization information to reflect the newly added keywords.

4.2.3 Functional Requirements

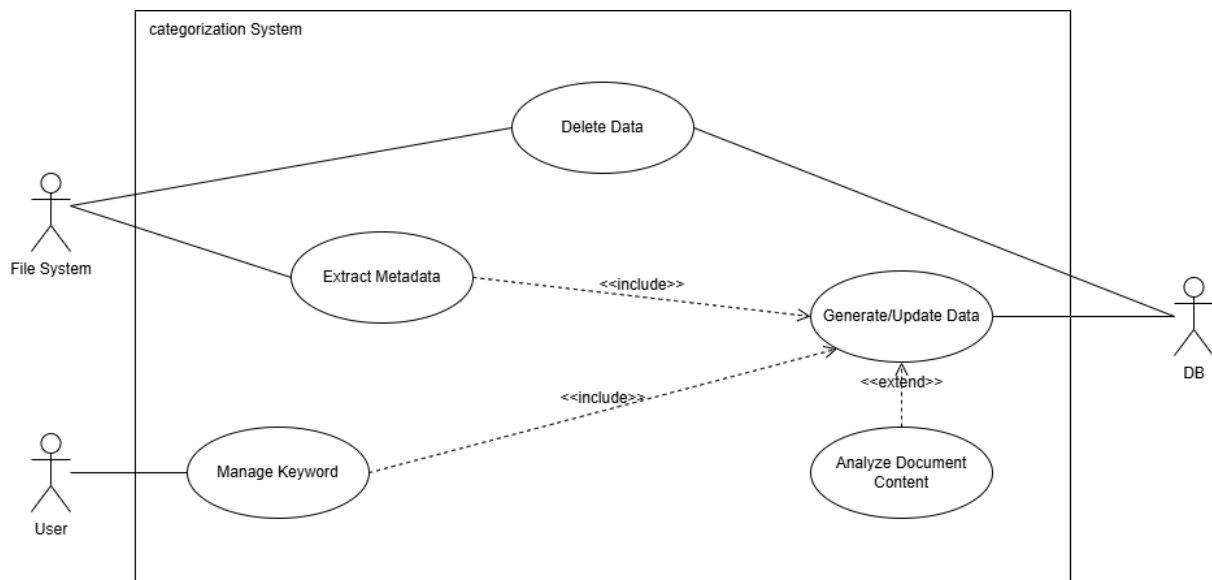
- 4.2.3.1 **[Watched Directory]:** The system shall allow users to register, modify, and remove watched directories to which automatic categorization is applied.
- 4.2.3.2 **[Event Detection]:** The system shall detect file creation and modification events in watched directories in real time.
- 4.2.3.3 **[Metadata Collection]:** The system shall collect metadata of detected files, including file name, full path, extension, creation date, modification date, and file size.
- 4.2.3.4 **[Primary Categorization]:** The system shall perform primary categorization based on metadata such as file extension, file name, storage location, and date information.
- 4.2.3.5 **[Text Analysis]:** The system shall extract and analyze text from supported document formats such as PDF, DOCX, and TXT.
- 4.2.3.6 **[Categorization Metadata Generation]:** The system shall generate internal categorization information such as subject, document type, and media type based on file metadata and content analysis results.
- 4.2.3.7 **[Categorization Metadata Storage]:** The system shall store generated categorization information in the local database and use it for search functions.
- 4.2.3.8 **[Keyword Management]:** The system shall allow users to add, modify, and delete one or more custom keywords for a specific file.
- 4.2.3.9 **[Keyword Reflection]:** The system shall store user-added keywords as part of the internal categorization information of the corresponding file.
- 4.2.3.10 **[Categorization Update]:** The system shall update the categorization information of a file when user-added keywords are

added or changed.

4.2.3.11 **[Search Reflection]**: The system shall reflect stored categorization information and user-added keywords in Smart Search query interpretation and result ranking.

4.2.3.12 **[Information Update]**: The system shall update existing categorization information when the file metadata, content, or user-added keywords are changed.

4.2.3.13 **[Exception Handling]**: The system shall perform metadata-based analysis only, or mark the file as an exception state, for unsupported, corrupted, locked, or unreadable file formats.



4.3 Personal Setting

4.3.1 Description

Personal Setting allows users to directly configure various elements of the software according to their preferences. The system must be able to modify its configuration based on user settings, and these changes must persist permanently.

During the initial installation, the software provides a setup wizard to specify the installation path, database storage location, target folders and extensions for Smart Search, and directories for Categorization.

After installation, users can modify these paths and UI elements, such as themes, through the settings interface. Users can also change the global hotkey for the Overlay UI, which the system must reflect immediately.

4.3.2 Stimulus/Response sequences

4.3.2.1 scenario 1: Initial Execution and Setup

- A. The user runs the app for the first time after install setup.
- B. The system displays the "Initial Setup Wizard" UI to configure paths and scopes
- C. The user selects the app installation path and the storage location for indexed data.
- D. The system verifies write permissions and creates the local DB structure.
- E. The user selects target extensions (e.g., .pdf, .txt, etc.) and directories for Smart Search
- F. The user selects target directories for Categorization.
- G. The system saves settings and simultaneously starts background keyword indexing, vectorization for the selected scope, and categorization of the current directories.

4.3.2.2 scenario 2: Changing Database Storage Location

- A. The user requests to modify the database storage path.
- B. The system verifies write permissions for the new directory, copies the existing DB to the new location, and performs indexing for any modified files.

4.3.2.3 scenario 3: Modifying Smart Search Scope

- A. The user changes the Smart Search target settings.
- B. The system immediately re-performs vectorization for the files within the newly defined scope.

4.3.2.4 scenario 4: Changing Categorization Directories

- A. The user modifies the target directories for categorization.
- B. The system removes existing categorization data for the old scope and restarts the process for the newly modified directories.

4.3.2.5 scenario 5: UI Theme Change

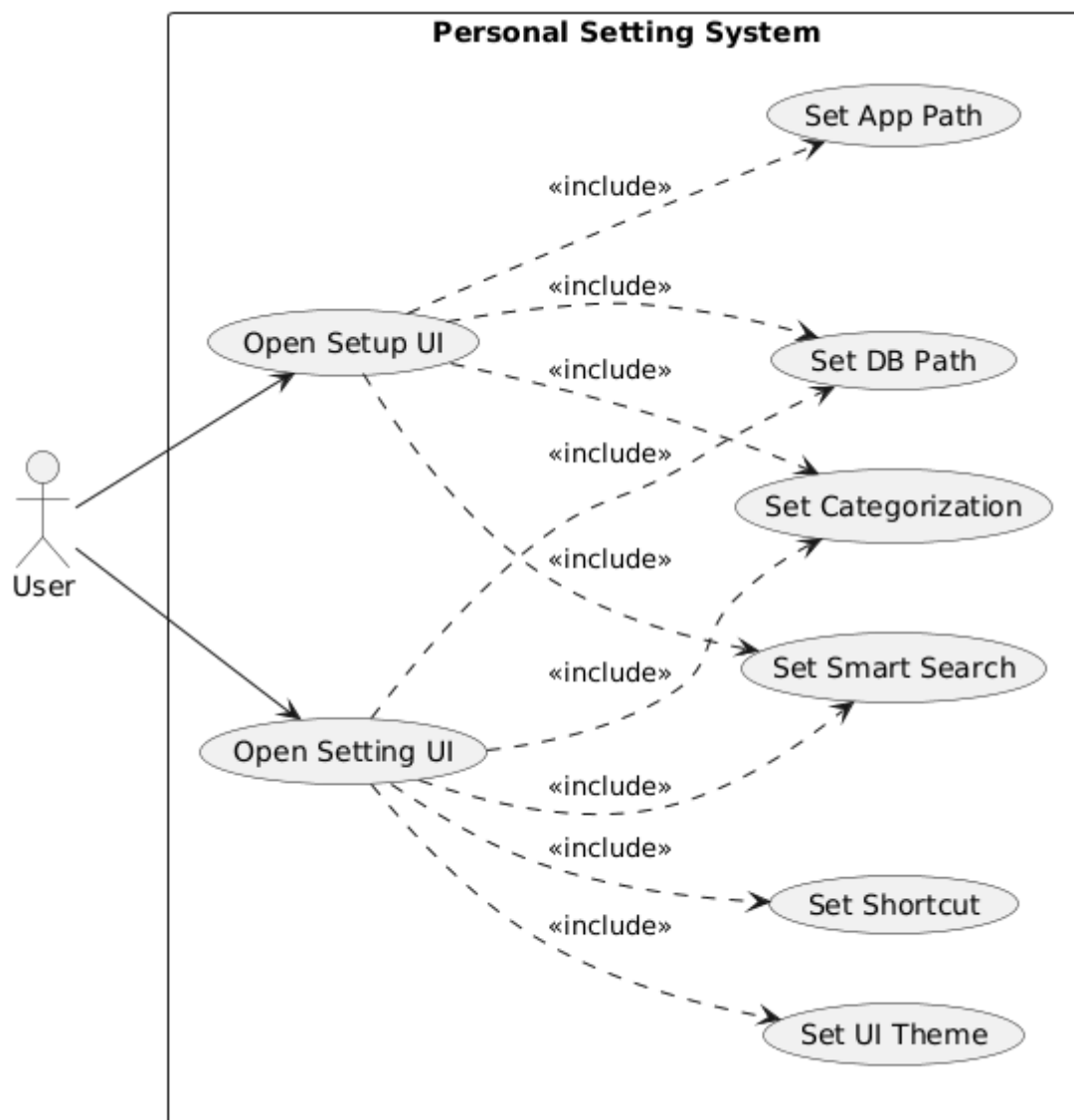
- A. The user switches the UI theme (e.g., Light Mode, Dark Mode)
- B. The system updates the UI data and reflects the theme change immediately.

4.3.2.6 scenario 6: Change global hotkey for Overlay UI

- A. The user changes the global hotkey for calling the Overlay UI.
- B. The system reflects and applies the new shortcut immediately.

4.3.3 Functional Requirements

- 4.3.3.1 **[Path Customization]**: The system must allow separate configurations for the app executable path and the database storage path during setup.
- 4.3.3.2 **[Search Target Configuration]**: The system must enable users to specify and change target directories and file extensions for Smart Search.
- 4.3.3.3 **[Categorization Target Configuration]**: The system must enable users to specify and change the target directories for categorization.
- 4.3.3.4 **[Setting Synchronization and Update]**: When non-UI settings are changed, the system must verify database integrity and immediately re-perform pre-processing for the modified scope.
- 4.3.3.5 **[UI Configuration and Refresh]**: The system must provide features to change UI settings and reflect those changes instantly.
- 4.3.3.6 **[Shortcut Modification]**: The system must allow users to change the global hotkey for the Overlay UI and apply it immediately.
- 4.3.3.7 **[Persistent Configuration Storage]**: All saved configuration information must be stored as a "config.json" file in the same directory as the application.



5 Non-functional Requirements

5.1 Performance Requirements

5.1.1 Search Latency: Quick search results should come out in 0.1 seconds.

5.1.2 Resource Usage: On background state, usage of the CPU must not exceed 5 percent.

5.2 Reliability

5.2.1 Software cannot change the contents of files that do not have write permission except DB and config.

5.3 Security

5.3.1 Local Storage: all databases must be saved in local storage system.

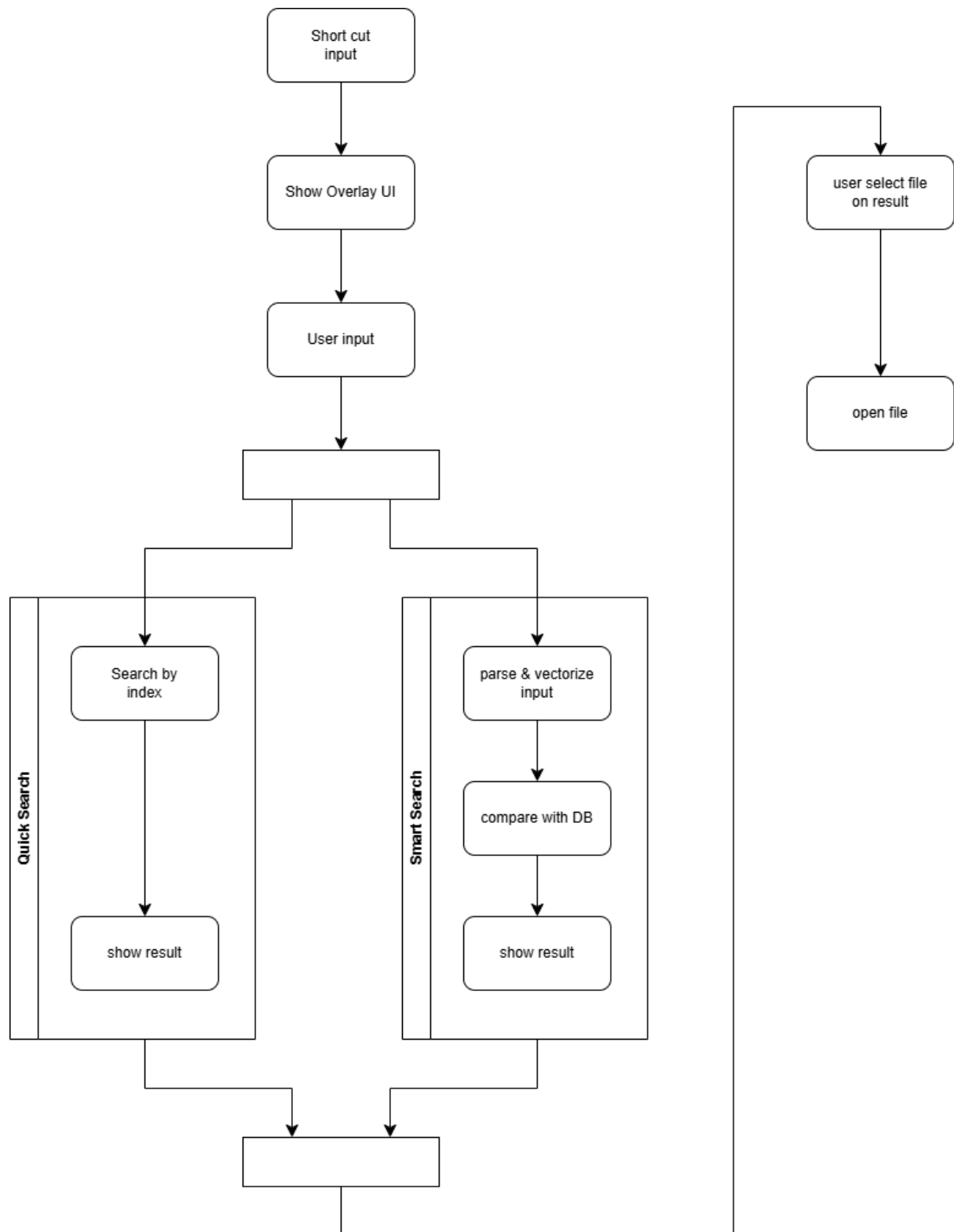
5.4 User Convenience

5.4.1 Users should not install additional features separately.

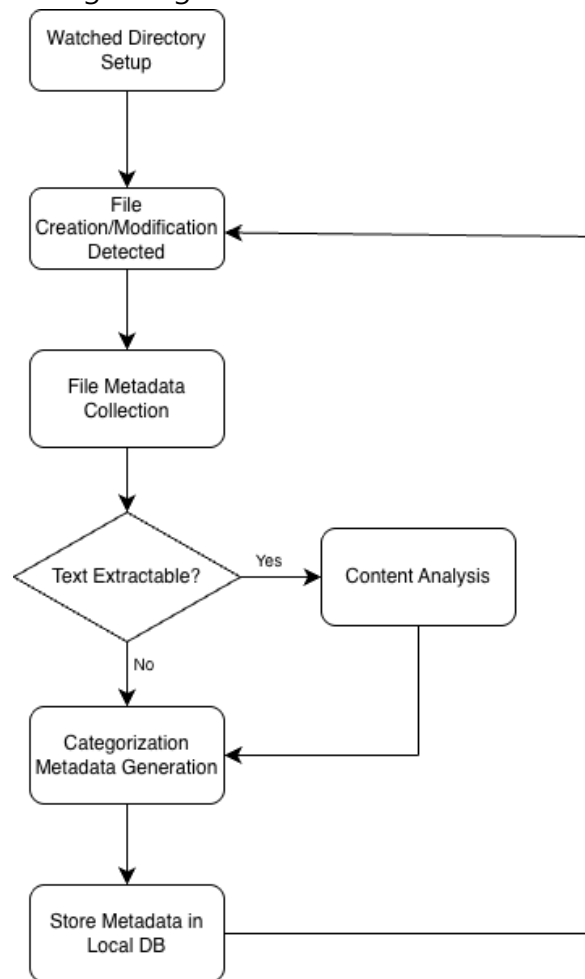
6 Appendix

6.1 Flow charts

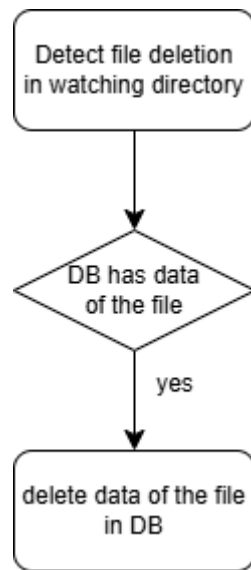
6.1.1 Search



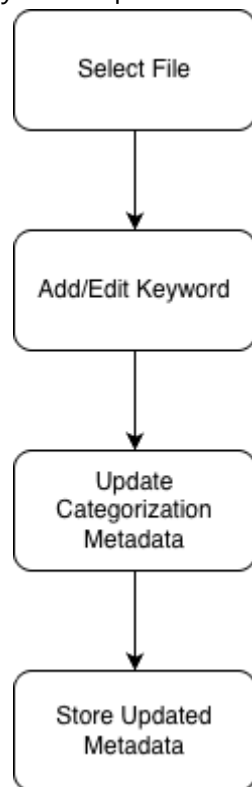
6.1.2 File Categorizing Process



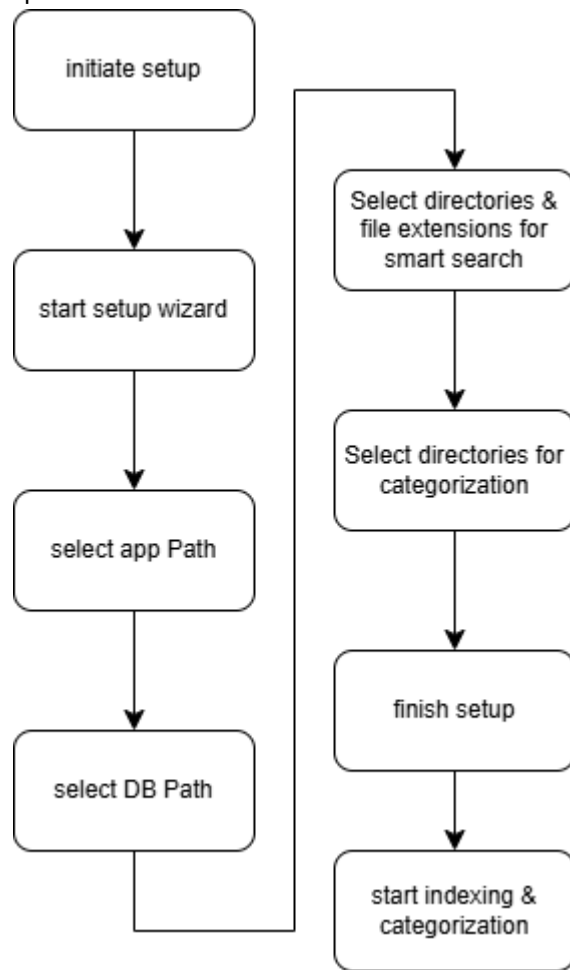
6.1.3 file deletion



6.1.4 Keyword Update Process



6.1.5 setup



6.1.6 change settings

